САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Фёдоров Иван

Группа К3340

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Цель лабораторной работы - реализовать backend-приложение с REST API на основе ранее подготовленных домашний заданий: предметной области, модели данных и спецификации API. В качестве предметной области выбран сервис аренды недвижимости RentEase.

Необходимо было выполнить следующие пункты:

1. Подготовить серверное приложение для обработки HTTP-запросов.
2. Реализовать основные REST-эндпоинты согласно OpenAPI-спецификации из ДЗ2.
3. Описать доменные сущности, соответствующие ERD и предметной области из ДЗ1.
4. Организовать работу с базой данных через ORM.
5. Добавить миграцию базы данных для создания начальной схемы.
6. Реализовать авторизацию пользователей и разграничение действий по ролям.
7. Проверить работу приложения через Swagger UI и тестовые HTTP-запросы.

В реализации был выбран Java-стек: Spring Boot вместо Node.js/Express. Такое решение сохраняет смысл задания: разработан backend с REST API, ORM, миграциями и проверяемыми пользовательскими сценариями.

Ход работы

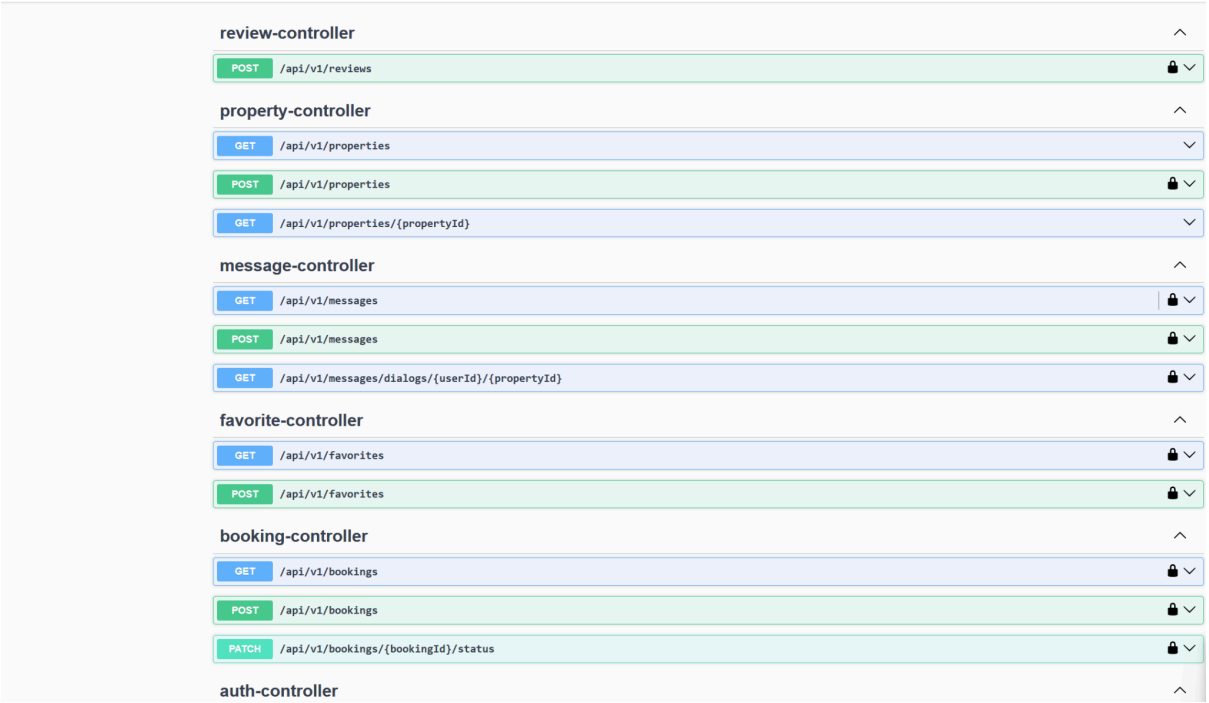
1. Анализ исходных материалов

Перед началом реализации были изучены материалы ДЗ1 и ДЗ2. В ДЗ1 была определена предметная область сервиса аренды недвижимости, ключевые роли пользователей и сущности системы. В ДЗ2 была подготовлена OpenAPI-спецификация, описывающая основные REST-эндпоинты.

Основные роли системы:

- tenant - арендатор, который ищет жилье и создает бронирования;
- landlord - арендодатель, который публикует объекты недвижимости и подтверждает заявки;
- admin - служебная административная роль.

Основные сущности системы: пользователь, объект недвижимости, бронирование, избранное, сообщение, договор аренды, платеж, отзыв.



OpenAPI-спецификация REST API сервиса RentEase

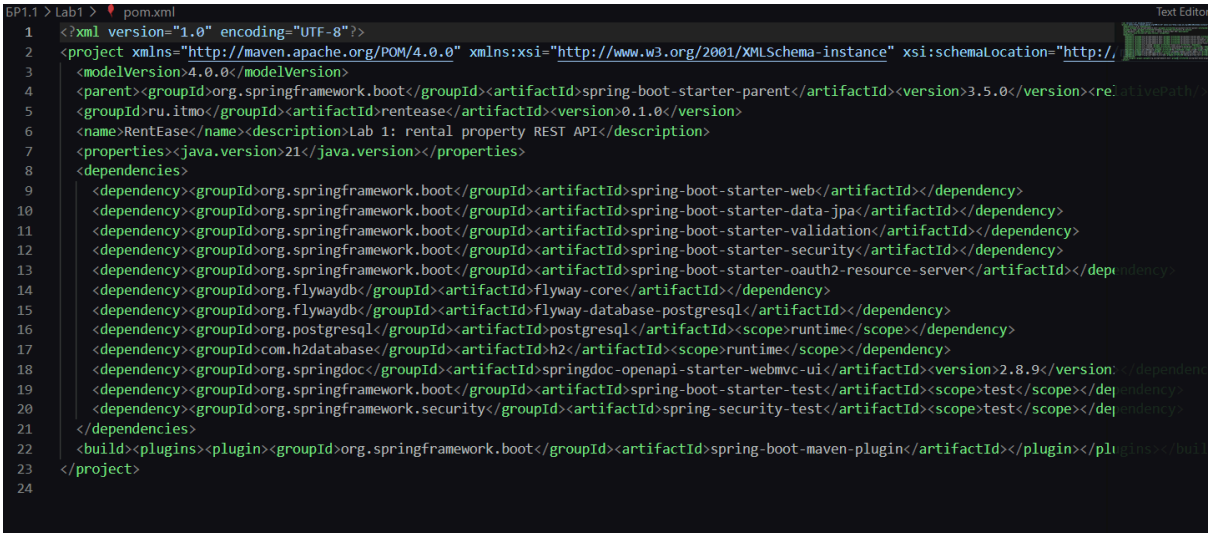
2. Создание Spring Boot проекта

Для реализации лабораторной работы был создан Maven-проект на Spring Boot. Конфигурация проекта находится в файле `pom.xml`. В нем подключены зависимости для web-сервера, REST API, JPA, валидации, безопасности, Flyway, PostgreSQL, H2 и Swagger UI.

Основные зависимости:

- `spring-boot-starter-web` - создание REST API и запуск встроенного Tomcat;
- `spring-boot-starter-data-jpa` - работа с базой данных через JPA;
- `spring-boot-starter-security` - настройка авторизации;
- `flyway-core` - применение миграций БД;
- `h2` - локальная файловая БД для проверки;
- `postgresql` - драйвер PostgreSQL для основного режима;
- `springdoc-openapi-starter-webmvc-ui` - Swagger UI.

Точка входа приложения находится в классе `RentEaseApplication.java`.



Maven-конфигурация Spring Boot проекта

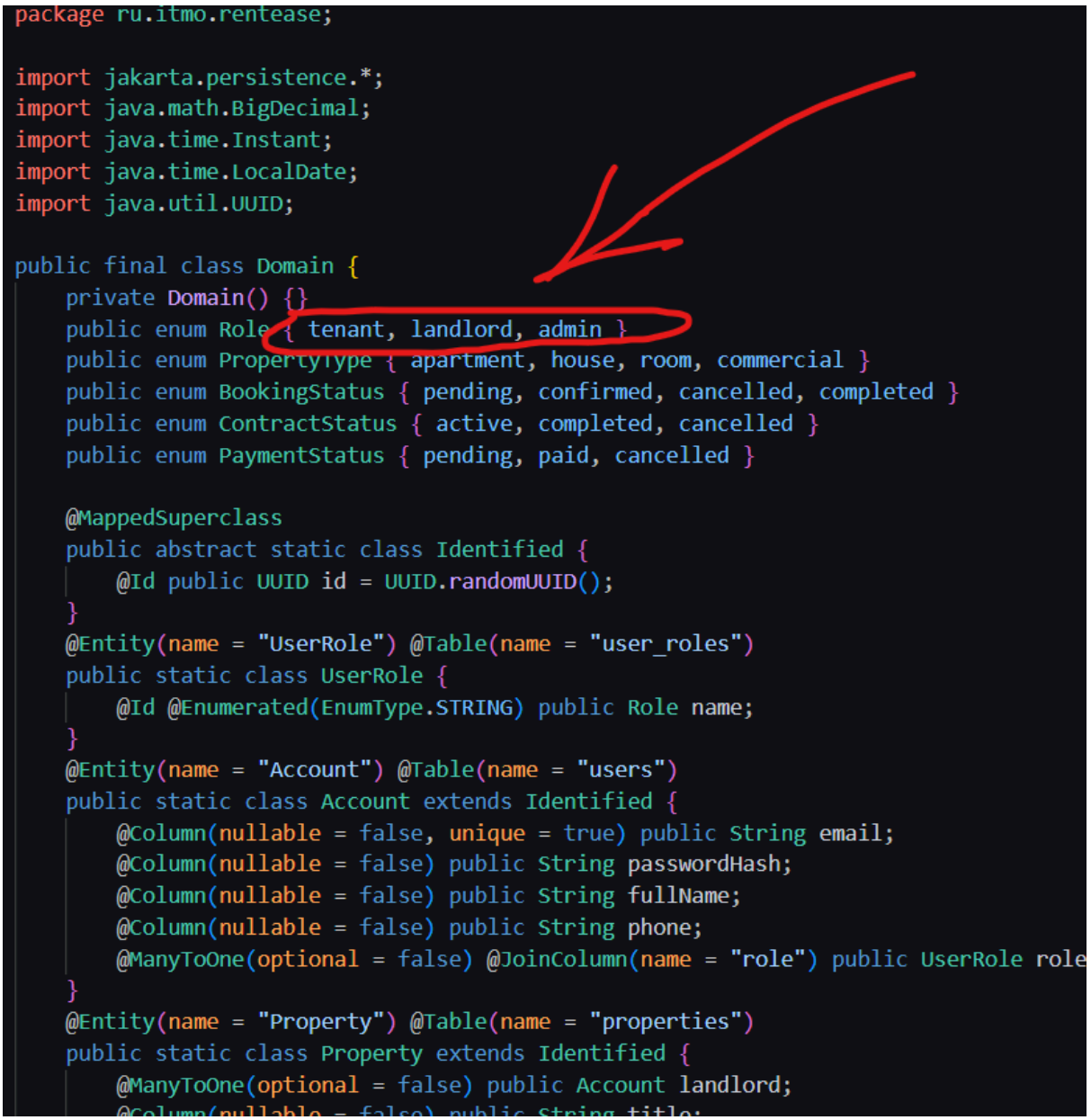
3. Реализация моделей предметной области

Доменные модели реализованы в файле `Domain.java`. Для связи Java-классов с таблицами базы данных используются JPA-аннотации: `@Entity`, `@Table`, `@Id`, `@Column`, `@ManyToOne`, `@OneToOne`.

Были реализованы следующие сущности:

- `Account`- пользователь системы;
- `UserRole`- роль пользователя;
- `Property`- объект недвижимости;
- `Booking`- бронирование;
- `Favorite`- избранный объект;
- `Message`- сообщение между пользователями;
- `RentalContract`- договор аренды;
- `Payment`- платеж;
- `Review`- отзыв.

Hibernate используется как реализация JPA: он читает аннотации сущностей и преобразует операции с Java-объектами в SQL-запросы к базе данных.



```java
package ru.itmo.rentease;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.Instant;
import java.time.LocalDate;
import java.util.UUID;

public final class Domain {
    private Domain() {}
    public enum Role { tenant, landlord, admin }
    public enum PropertyType { apartment, house, room, commercial }
    public enum BookingStatus { pending, confirmed, cancelled, completed }
    public enum ContractStatus { active, completed, cancelled }
    public enum PaymentStatus { pending, paid, cancelled }

    @MappedSuperclass
    public abstract static class Identified {
        @Id public UUID id = UUID.randomUUID();
    }
    @Entity(name = "UserRole") @Table(name = "user_roles")
    public static class UserRole {
        @Id @Enumerated(EnumType.STRING) public Role name;
    }
    @Entity(name = "Account") @Table(name = "users")
    public static class Account extends Identified {
        @Column(nullable = false, unique = true) public String email;
        @Column(nullable = false) public String passwordHash;
        @Column(nullable = false) public String fullName;
        @Column(nullable = false) public String phone;
        @ManyToOne(optional = false) @JoinColumn(name = "role") public UserRole role
    }
    @Entity(name = "Property") @Table(name = "properties")
    public static class Property extends Identified {
        @ManyToOne(optional = false) public Account landlord;
```

JPA-сущности предметной области RentEase

4. Настройка базы данных и миграций Flyway

Начальная структура базы данных описана в миграции `V1__initial_schema.sql`. Файл находится в каталоге `src/main/resources/db/migration`.

При запуске приложения Flyway автоматически применяет миграцию и создает таблицы: `users`, `user_roles`, `properties`, `bookings`, `favorites`, `messages`, `rental_contracts`, `payments`, `reviews`.

Для локальной проверки используется файловая база H2, настроенная в `application-local.properties`. Данные локальной БД хранятся в

файле `Lab1/data/rentease.mv.db`. Информацию о примененных миграциях Flyway хранит в служебной таблице `flyway_schema_history`.

В основном профиле приложение рассчитано на PostgreSQL. Настройки подключения находятся в `application.properties`.

```
CREATE TABLE user_roles (name varchar(255) PRIMARY KEY);
INSERT INTO user_roles(name) VALUES ('tenant'), ('landlord'), ('admin');
CREATE TABLE users (
 id uuid PRIMARY KEY, email varchar(255) NOT NULL UNIQUE, password_hash varchar(255) NOT NULL,
 full_name varchar(255) NOT NULL, phone varchar(255) NOT NULL, role varchar(255) NOT NULL REFERENCES
);
CREATE TABLE properties (
 id uuid PRIMARY KEY, landlord_id uuid NOT NULL REFERENCES users(id), title varchar(255) NOT NULL,
 description varchar(10000) NOT NULL, property_type varchar(255) NOT NULL CHECK (property_type IN ('
 price_per_month numeric(14,2) NOT NULL CHECK (price_per_month > 0), security_deposit numeric(14,2)
 city varchar(255) NOT NULL, address varchar(255) NOT NULL, rooms_count integer CHECK (rooms_count >
 area_sqm numeric(12,2), is_active boolean NOT NULL
);
CREATE INDEX properties_search_idx ON properties(is_active, city, property_type, price_per_month);
CREATE TABLE rental_conditions (
 id uuid PRIMARY KEY, property_id uuid NOT NULL UNIQUE REFERENCES properties(id), minimum_months int
 utilities_included boolean NOT NULL, pets_allowed boolean NOT NULL, notes varchar(2000)
);
CREATE TABLE property_photos (
 id uuid PRIMARY KEY, property_id uuid NOT NULL REFERENCES properties(id), content_type varchar(255)
);
CREATE TABLE favorites (
 id uuid PRIMARY KEY, user_id uuid NOT NULL REFERENCES users(id), property_id uuid NOT NULL REFERENC
);
CREATE TABLE bookings (
 id uuid PRIMARY KEY, tenant_id uuid NOT NULL REFERENCES users(id), property_id uuid NOT NULL REFERE
 start_date date NOT NULL, end_date date NOT NULL, status varchar(255) NOT NULL, message varchar(400
 CHECK(end_date >= start_date), CHECK(status IN ('pending','confirmed','cancelled','completed'))
);
CREATE INDEX bookings_dates_idx ON bookings(property_id, status, start_date, end_date);
CREATE TABLE rental_contracts (
 id uuid PRIMARY KEY, booking_id uuid NOT NULL UNIQUE REFERENCES bookings(id), contract_number varch
 monthly_rent numeric(14,2) NOT NULL, status varchar(255) NOT NULL CHECK(status IN ('active','comple
);
CREATE TABLE payments (
```

SQL-миграция Flyway для начальной схемы БД

5. Реализация репозиториев

Для доступа к данным использован Spring Data JPA. Репозитории находятся в пакете `ru.itmo.rentease.repo`. Они наследуются от `JpaRepository`и позволяют выполнять стандартные операции без ручного SQL: поиск, сохранение, удаление и получение списков.

Также были добавлены методы поиска по бизнес-условиям. Например, в `BookingRepository`реализован поиск бронирований арендатора, поиск

заявок по арендодателю и проверка пересечения дат бронирования.

```java
package ru.itmo.rentease.repo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import ru.itmo.rentease.Domain.*;
import java.util.List;
import java.util.UUID;

public interface BookingRepository extends JpaRepository<Booking, UUID> {
    List<Booking> findByTenantId(UUID tenantId);
    List<Booking> findByTenantIdAndStatus(UUID tenantId, BookingStatus status);
    @Query("SELECT b FROM Booking b WHERE b.property.landlord.id = :landlordId")
    List<Booking> findByLandlordId(UUID landlordId);
    @Query("SELECT b FROM Booking b WHERE b.property.landlord.id = :landlordId AND b.status = :status")
    List<Booking> findByLandlordIdAndStatus(UUID landlordId, BookingStatus status);

    @Query("""
        SELECT COUNT(b) > 0 FROM Booking b
        WHERE b.property.id = :propertyId
          AND b.status IN :statuses
          AND b.startDate <= :endDate
          AND b.endDate >= :startDate
        """)
    boolean existsOverlappingBooking(UUID propertyId, java.time.LocalDate startDate, java.time.LocalDate
                                     List<BookingStatus> statuses);
}
```

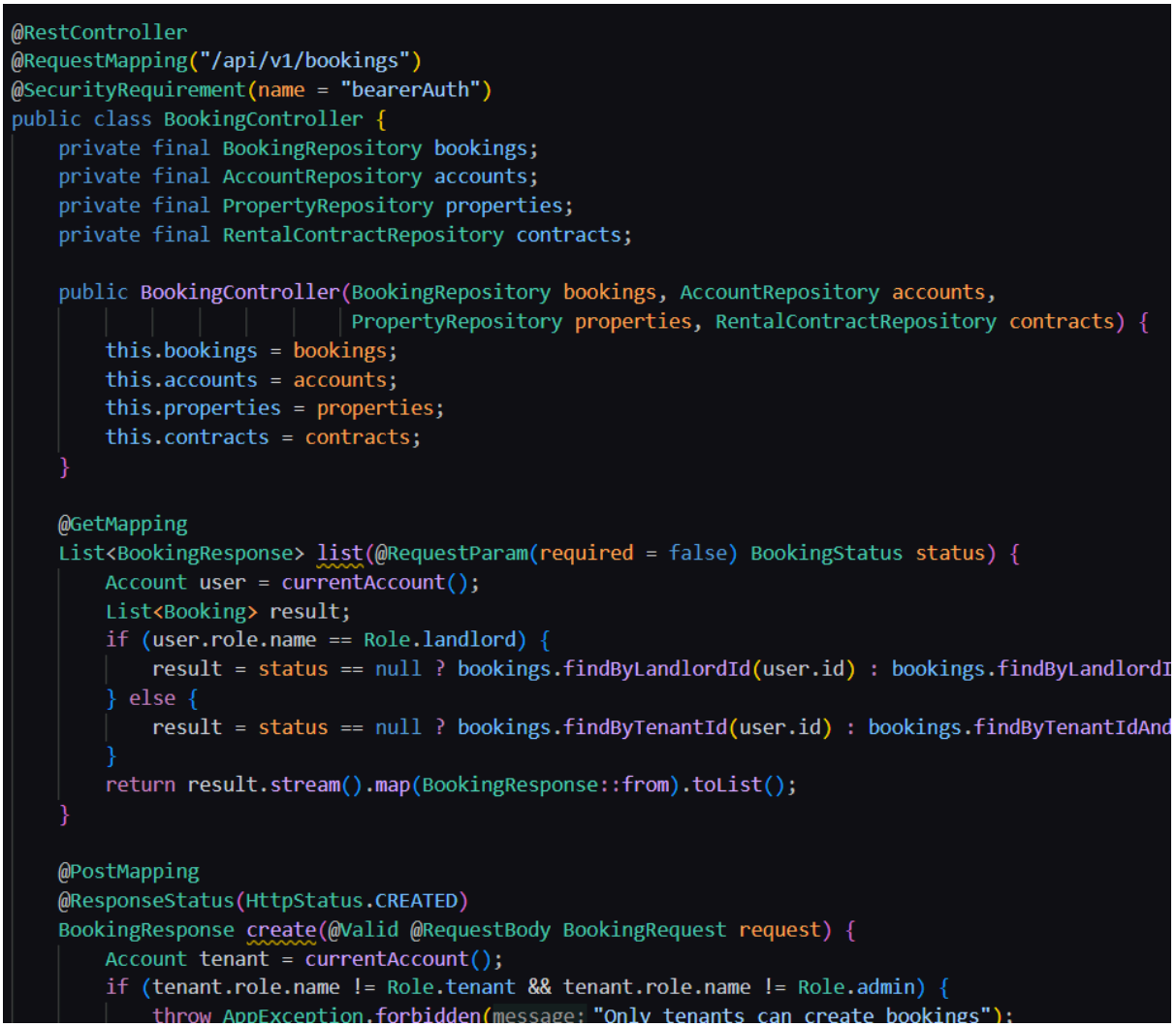
Репозиторий бронирований на Spring Data JPA

6. Реализация REST-контроллеров

Контроллеры принимают HTTP-запросы, проверяют входные данные, вызывают репозитории и возвращают JSON-ответы. Были реализованы следующие контроллеры:

- `AuthController` - регистрация, вход, получение текущего пользователя;
- `PropertyController` - просмотр и создание объектов недвижимости;
- `BookingController` - создание, просмотр и изменение статуса бронирований;
- `FavoriteController` - работа с избранными объектами;
- `MessageController` - отправка и просмотр сообщений;
- `ContractController` - просмотр договоров;
- `ReviewController` - создание отзывов.

REST API использует стандартные HTTP-методы: `GET` для получения данных, `POST` для создания сущностей, `PATCH` для частичного изменения данных. Формат обмена данными - JSON.



```
@RestController
@RequestMapping("/api/v1/bookings")
@SecurityRequirement(name = "bearerAuth")
public class BookingController {
    private final BookingRepository bookings;
    private final AccountRepository accounts;
    private final PropertyRepository properties;
    private final RentalContractRepository contracts;

    public BookingController(BookingRepository bookings, AccountRepository accounts,
                             PropertyRepository properties, RentalContractRepository contracts) {
        this.bookings = bookings;
        this.accounts = accounts;
        this.properties = properties;
        this.contracts = contracts;
    }

    @GetMapping
    List<BookingResponse> list(@RequestParam(required = false) BookingStatus status) {
        Account user = currentAccount();
        List<Booking> result;
        if (user.role.name == Role.landlord) {
            result = status == null ? bookings.findByLandlordId(user.id) : bookings.findByLandlordI
        } else {
            result = status == null ? bookings.findByTenantId(user.id) : bookings.findByTenantIdAnd
        }
        return result.stream().map(BookingResponse::from).toList();
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    BookingResponse create(@Valid @RequestBody BookingRequest request) {
        Account tenant = currentAccount();
        if (tenant.role.name != Role.tenant && tenant.role.name != Role.admin) {
            throw AppException.forbidden(message: "Only tenants can create bookings");
```

REST-контроллер для работы с бронированиями

7. Реализация авторизации JWT

Для защищенных эндпоинтов реализована авторизация через JWT. При успешном входе пользователь получает `access_token`, `refresh_token`и `token_type`. Для вызова защищенных методов клиент передает токен в заголовке `Authorization: Bearer <access_token>`.

Класс `SecurityConfig.java`настраивает правила доступа. Публичными являются health-check, регистрация, вход, Swagger UI и просмотр объектов недвижимости. Остальные запросы требуют JWT.

Дополнительно реализованы проверки ролей:

- создавать объявления может только `landlord`или `admin`;
- создавать бронирования может только `tenant`или `admin`;
- подтверждать бронирование может арендодатель своего объекта или `admin`;

- отзыв может оставить участник договора или `admin`.

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http, JwtFilter jwtFilter) throws Exception {
        return http
                .csrf(csrf -> csrf.disable())
                .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STA
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers(
                                "/api/v1/health",
                                "/api/v1/auth/**",
                                "/docs",
                                "/docs/**",
                                "/swagger-ui.html",
                                "/swagger-ui/**",
                                "/v3/api-docs",
                                "/v3/api-docs/**"
                        ).permitAll()
                        .requestMatchers(org.springframework.http.HttpMethod.GET, "/api/v1/properties
                        .anyRequest().authenticated())
                .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
                .build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
```

```java
@Component
static class JwtFilter extends OncePerRequestFilter {
    private final JwtService jwtService;

    JwtFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, Fil
            throws ServletException, IOException {
        String header = request.getHeader(name: "Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                JwtService.JwtUser user = jwtService.parse(header.substring(7));
                var auth = new UsernamePasswordAuthenticationToken(
                        user.userId(), null, List.of(new SimpleGrantedAuthority("ROLE_" + user.ro
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (AppException ex) {
                response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
                response.setContentType(type: "application/json");
                response.getWriter().write("{\"code\":\"UNAUTHORIZED\",\"message\":\"Invalid toke
                return;
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

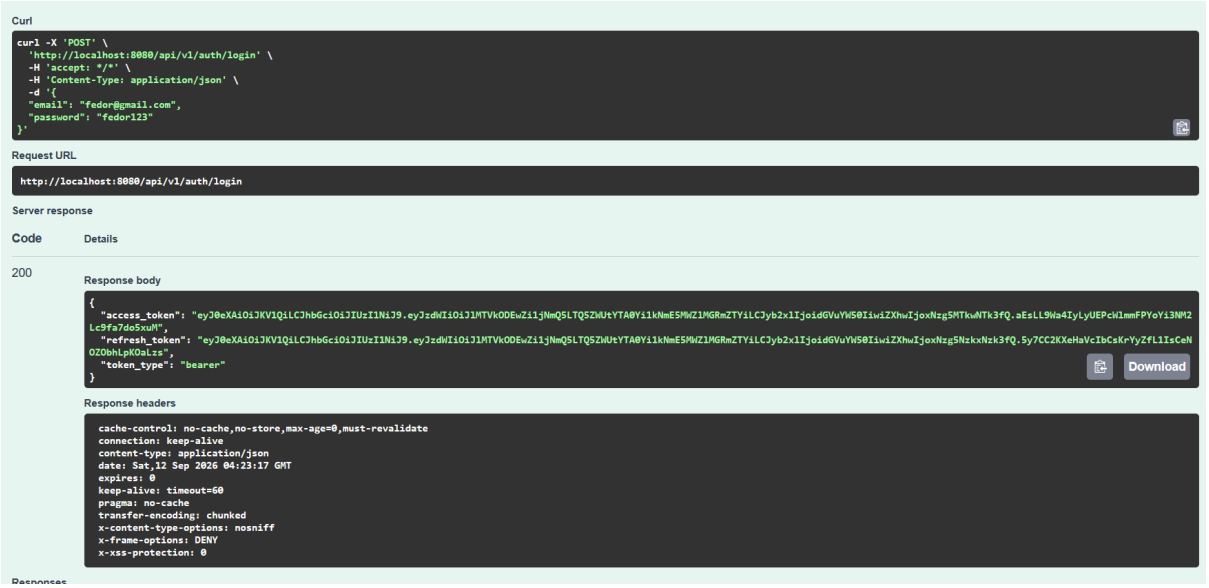
Настройка JWT-защиты и публичных маршрутов

8. Проверка работы через Swagger UI

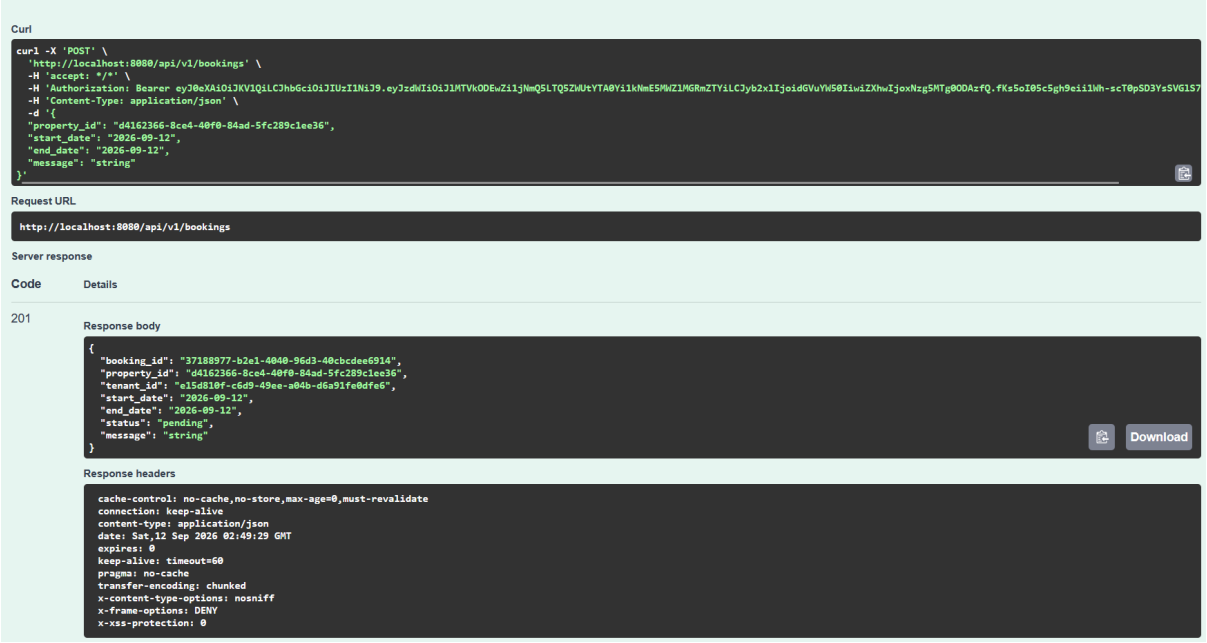
Для проверки API использовался Swagger UI по адресу `http://localhost:8080/docs`. Через Swagger были выполнены основные запросы: регистрация, вход, создание объекта недвижимости, создание бронирования, подтверждение бронирования и получение договора.

Основной smoke-test состоял из следующих шагов:

1. Проверка `GET /api/v1/health`.
2. Регистрация пользователя с ролью `landlord`.
3. Авторизация `landlord`и получение JWT.
4. Создание объекта недвижимости через `POST /api/v1/properties`.
5. Регистрация пользователя с ролью `tenant`.
6. Авторизация `tenant`и получение JWT.
7. Создание бронирования через `POST /api/v1/bookings`.
8. Подтверждение бронирования арендодателем через `PATCH /api/v1/bookings/bookingId/status`.
9. Проверка созданного договора через `GET /api/v1/contracts`.



Получение JWT-токена после авторизации



Создание бронирования арендатором



Автоматически созданный договор аренды

Вывод

В ходе лабораторной работы был реализован backend-сервис RentEase для аренды недвижимости. Реализация основана на результатах ДЗ1 и ДЗ2: предметной области, модели данных и OpenAPI-спецификации.

В проекте реализованы доменные сущности, REST-контроллеры, репозитории Spring Data JPA, JWT-авторизация, миграция базы данных Flyway и Swagger UI для проверки API. Приложение поддерживает основные сценарии: регистрацию пользователей, вход в систему, просмотр и создание объектов недвижимости, создание и подтверждение бронирований, создание договоров, работу с избранным, сообщениями и

отзывами.

Локальная проверка показала, что приложение запускается, Swagger UI доступен, основные HTTP-запросы выполняются корректно, а данные сохраняются в локальной базе H2. Таким образом, требования лабораторной работы по реализации REST API, работе с ORM и базой данных выполнены.